UNIVERSITI TEKNOLOGI MALAYSIA

POST MODULE ASSESSMENT

SEMESTER 2, SESSION 2025/2026

Comparative Study of Fruit Basket Image Classification and Object Detection Using Python

Name:	Diviyah Kumara Rajah
Matric Number :	MRT254029
Course Code:	MRTB2153 - Advanced Artificial Intelligence
Section:	MRTB2153-50, Semester 2, Session 2025/2026
Lecturer	Mohd Syahid bin Mohd Anuar
Link to Google Colab:	https://colab.research.google.com/drive/11E_o_8YQ041X50ryn1DQvktvoWfZwG10?usp=sharing

Table of Contents

Table of Contents 2
1. Introduction 3
2. Image Classification 3
2.1 Data Preparation 3
Directory Structure 4
2.2 Data Visualisation 4
2.3 Model Design and Results 4
2.3.1 Model 1: Support Vector Machine (SVM) 5
2.3.2 Model 2: Custom CNN 5
2.3.3 Model 3: MobileNetV2 (Transfer Learning) 6
2.4 Classification Discussion 7
3. Object Detection 9
3.1 Data Preparation 9
3.1.1 Annotation Format and Conversion 9
3.1.2 Class Filtering and Basket Image Separation 9
3.1.3 Directory Structure 10
3.2 Object Detection Strategy 10
3.3 Results Visualisation 10
3.4.1 YOLOv8n 10
3.4.2 SSD-Lite (MobileNetV3) 11
3.4.3 Faster R-CNN (ResNet50-FPN) 12
4. Critical Comparative Analysis and Discussion 13
4.1 Accuracy vs. Speed Trade-offs 13
4.2 Model Complexity and Training Difficulty 14
4.3 Final Recommendation 14
5. Conclusion 16

1. Introduction

This report presents a comparative study of two computer vision tasks performed on a custom Apple vs. Orange dataset:

(1) image classification using three distinct modelling approaches

(2) object detection using three distinct deep learning detector architectures.

The aim of this project is to design, implement, and critically compare these models using both quantitative metrics and qualitative real-world testing, and to derive recommendations for practical deployment scenarios.

Apple and Orange were selected as the target fruit classes for this study. Apples and oranges share a similar round shape and overall size, which makes shape-based discrimination alone unreliable. However, the two fruits differ significantly in surface colour (red/green for apples versus orange for oranges) and surface texture (smooth, waxy skin for apples versus a dimpled rind for oranges). This combination makes colour and texture the primary discriminative features, allowing this study to examine whether the implemented models rely on colour cues alone or learn more robust, combined feature representations.

2. Image Classification

2.1 Data Preparation

The classification dataset was sourced from the Fruits-360 dataset (Kaggle: moltean/fruits), accessed programmatically via the kagglehub library within Google Colab. The 100x100 pixel resolution variant of the dataset (fruits-360_100x100) was used.

The raw dataset contains many fine-grained sub-variety folders for each fruit (for example, “Apple Red 1”, “Apple Golden 1”, “Apple Braeburn 1”, “Orange 1”). A custom mapping dictionary was constructed to merge selected, genuine apple and orange sub-varieties into two unified classes, while explicitly excluding visually similar but unrelated folders such as “Tomato Cherry Orange” and “Pepper Orange”, which contain the word “Orange” in their folder name but are not true oranges. This filtering step was essential to ensure label correctness.

The following Apple and Orange sub-varieties were selected and aggregated:

- Apple: Apple Red 1, Apple Red 2, Apple Red 3, Apple Golden 1, Apple Braeburn 1, Apple Granny Smith 1
- Orange: Orange 1, Orange 2, Orange 3

All aggregated images were shuffled (with a fixed random seed of 42 for reproducibility) and split into Train, Validation, and Test sets using a 70% / 15% / 15% ratio. The resulting dataset distribution is summarised in Table 2.1.

Split	Apple	Orange	Total
Train	2013	1327	3340

Validation	431	284	715
Test	433	286	719
Total	2877	1897	4774

Table 2.1: Classification dataset distribution across Train/Validation/Test splits.

It is noted that the resulting dataset is moderately imbalanced (approximately 60% Apple images versus 40% Orange images). This imbalance was carried forward intentionally rather than artificially corrected, and per-class precision/recall (rather than accuracy alone) was used during evaluation to ensure the imbalance did not mask poor performance on the minority class.

Directory Structure

Images were copied into a directory structure organised by split and class, as illustrated below, with each filename prefixed by its original sub-variety folder name to avoid filename collisions during merging:

```
classification_processed/
  train/
    Apple/
    Orange/
  val/
    Apple/
    Orange/
  test/
    Apple/
    Orange/
```

2.2 Data Visualisation

Prior to model training, sample images from each class and basic class-distribution statistics were visualised to confirm correct labelling and to inspect visual characteristics relevant to feature engineering (colour and texture).

[PASTE HERE: Sample grid of Apple vs. Orange training images]

Figure 2.1: Sample images from the Apple and Orange classes.

[PASTE HERE: Bar chart of class distribution (Train/Val/Test) from Table 2.1]

Figure 2.2: Class distribution across dataset splits.

2.3 Model Design and Results

Three distinct classification approaches were implemented and evaluated on the identical held-out test set (719 images): a traditional machine learning model (Support Vector Machine), a Convolutional Neural Network trained from scratch, and a transfer learning model (MobileNetV2). Table 2.2 summarises the comparative performance of all three models.

Model	Test Accuracy	Precision	Recall	F1-Score	Trainable Params
SVM (HOG + HSV Histogram)	1.00	1.00	1.00	1.00	N/A (classical ML)
Custom CNN (3-layer)	1.00	1.00	1.00	1.00	3,304,769
MobileNetV2 (Transfer Learning)	1.00	1.00	1.00	1.00	1,281 (frozen backbone)

Table 2.2: Comparative performance of the three image classification models on the test set.

2.3.1 Model 1: Support Vector Machine (SVM)

The first model implemented a traditional, pre-deep-learning computer vision pipeline. Each image was resized to 128×128 pixels, and two handcrafted feature types were extracted: a HSV colour histogram (capturing the dominant colour profile of each fruit) and a Histogram of Oriented Gradients (HOG) descriptor (capturing edge and texture structure). The concatenated feature vector was standardised using StandardScaler and used to train a Support Vector Machine with a Radial Basis Function (RBF) kernel.

The SVM achieved a test accuracy of 100% perfect result demonstrates that, under the controlled conditions of the Fruits-360 dataset, handcrafted colour and texture features are sufficient to almost perfectly separate the two classes, validating the visual-property justification for selecting Apple and Orange as distinct classes.

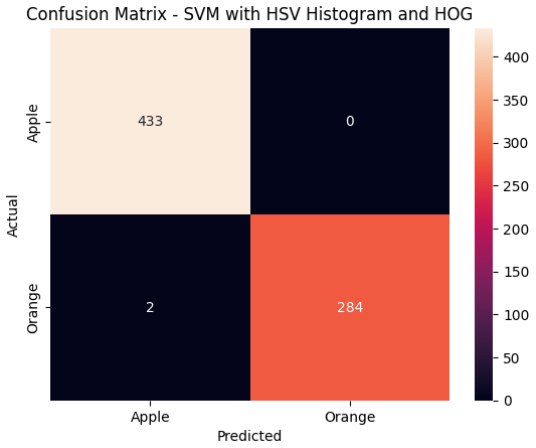

Figure 2.3: Confusion Matrix for the SVM model on the test set.

2.3.2 Model 2: Custom CNN

The second model was a lightweight Convolutional Neural Network trained entirely from scratch, consisting of three convolutional blocks (32, 64, and 128 filters respectively, each followed by max-pooling), a fully connected layer of 128 units with 50% dropout for regularisation, and a sigmoid output layer for binary classification. Unlike the SVM, this model learned its own feature representations directly from **raw pixel data** rather than relying on handcrafted descriptors.

The CNN achieved 100% accuracy on the held-out test set, with zero misclassifications across all 719 images. Examination of the training curves shows a brief instability around epoch 7–8 (a small accuracy dip and corresponding loss spike) before the model recovered and converged to near-zero loss by epoch 10. This is consistent with the small dataset and relatively aggressive learning dynamics of training a CNN from scratch.

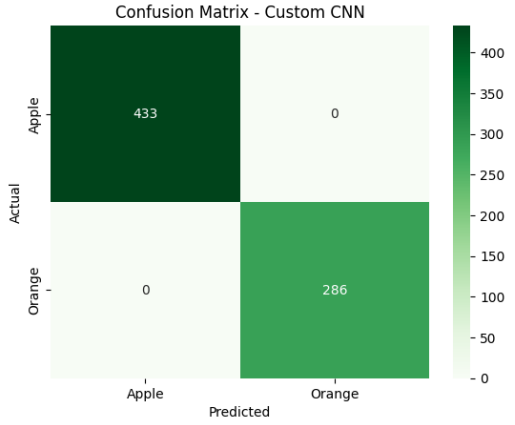

Figure 2.4: Confusion Matrix for the Custom CNN model on the test set.

```
Total params: 3,304,769 (12.61 MB)
Trainable params: 3,304,769 (12.61 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 22s 150ms/step - accuracy: 0.9368 - loss: 0.1424 - val_accuracy: 1.0000 - val_loss: 3.3432e-04
Epoch 2/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.9958 - loss: 0.0095 - val_accuracy: 1.0000 - val_loss: 2.2372e-04
Epoch 3/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 0.9967 - loss: 0.0065 - val_accuracy: 1.0000 - val_loss: 2.7199e-05
Epoch 4/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 1.0000 - loss: 6.5748e-04 - val_accuracy: 1.0000 - val_loss: 5.7677e-6
Epoch 5/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 1.0000 - loss: 0.0011 - val_accuracy: 1.0000 - val_loss: 3.1499e-07
Epoch 6/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 1.0000 - loss: 5.5330e-04 - val_accuracy: 1.0000 - val_loss: 1.7821e-6
Epoch 7/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 1.0000 - loss: 6.9817e-05 - val_accuracy: 1.0000 - val_loss: 1.8411e-6
Epoch 8/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 17ms/step - accuracy: 1.0000 - loss: 9.0748e-05 - val_accuracy: 1.0000 - val_loss: 3.5137e-6
Epoch 9/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 18ms/step - accuracy: 1.0000 - loss: 2.4759e-04 - val_accuracy: 1.0000 - val_loss: 1.5765e-6
Epoch 10/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 19ms/step - accuracy: 1.0000 - loss: 2.4038e-04 - val_accuracy: 1.0000 - val_loss: 3.0403e-1
```

Figure 2.5: Training and validation accuracy/loss curves for the Custom CNN.

2.3.3 Model 3: MobileNetV2 (Transfer Learning)

The third model used transfer learning, leveraging MobileNetV2 pre-trained on the ImageNet dataset. The convolutional base was frozen (non-trainable), and only a lightweight classification head, a GlobalAveragePooling2D layer, dropout, and a single dense sigmoid output was trained on the Apple/Orange dataset. This design choice means only 1,281 parameters were updated during training, compared to over 3.3 million for the CNN trained from scratch.

MobileNetV2 also achieved 100% test accuracy, with training converging considerably faster and more smoothly than the from-scratch CNN, reaching near-zero validation loss within the first 2–3 epochs. This demonstrates the practical efficiency benefit of transfer learning: equivalent predictive performance was achieved while training over 2,500 times fewer parameters.

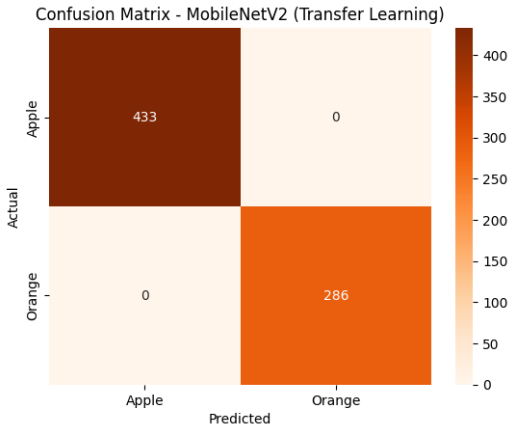

Figure 2.6: Confusion Matrix for the MobileNetV2 model on the test set.

```
Total params: 2,259,265 (8.62 MB)
Trainable params: 1,281 (5.00 KB)
Non-trainable params: 2,257,984 (8.61 MB)
Epoch 1/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 53s 344ms/step - accuracy: 0.9467 - loss: 0.1731 - val_accuracy: 1.0000 - val_loss: 0.0333
Epoch 2/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.9997 - loss: 0.0261 - val_accuracy: 1.0000 - val_loss: 0.0134
Epoch 3/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 1.0000 - loss: 0.0131 - val_accuracy: 1.0000 - val_loss: 0.0076
Epoch 4/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 1.0000 - loss: 0.0084 - val_accuracy: 1.0000 - val_loss: 0.0050
Epoch 5/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 1.0000 - loss: 0.0060 - val_accuracy: 1.0000 - val_loss: 0.0035
Epoch 6/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 1.0000 - loss: 0.0044 - val_accuracy: 1.0000 - val_loss: 0.0027
Epoch 7/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 1.0000 - loss: 0.0036 - val_accuracy: 1.0000 - val_loss: 0.0021
Epoch 8/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 2s 15ms/step - accuracy: 1.0000 - loss: 0.0029 - val_accuracy: 1.0000 - val_loss: 0.0017
Epoch 9/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 14ms/step - accuracy: 1.0000 - loss: 0.0024 - val_accuracy: 1.0000 - val_loss: 0.0014
Epoch 10/10
105/105 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 1.0000 - loss: 0.0020 - val_accuracy: 1.0000 - val_loss: 0.0012
```

Figure 2.7: Training and validation accuracy/loss curves for MobileNetV2.

2.4 Classification Discussion

All three classification models achieved perfect accuracy on the held-out test set. This result is attributable to the Fruits-360 dataset's controlled studio conditions , a uniform white background, consistent lighting, and a centred, single fruit per image combined with the strong colour separability between apples (red/green) and oranges (orange). Under these conditions, even simple handcrafted colour-based features are sufficient to almost perfectly distinguish the two classes, which explains why the SVM, despite having no learned feature representation, performed almost as well as the two deep learning models.

More importantly, MobileNetV2 achieved equivalent performance to the CNN trained from scratch while training only 0.04% of the parameters (1,281 vs. 3,304,769), converging in fewer epochs, clearly demonstrating the efficiency advantage of transfer learning when a suitable pre-trained backbone is available.

This classification performance should not be interpreted as evidence that these models would generalise equally well to real-world, cluttered scenes. This is precisely why the object detection task in

Section 3, which uses cluttered, multi-fruit basket images rather than clean studio photographs, is necessary to test the true robustness of learned visual features.

3. Object Detection

3.1 Data Preparation

The object detection dataset was sourced from a separate Kaggle dataset, “Fruit Images for Object Detection” (mbkinaci/fruit-images-for-object-detection), which provides 240 training images and 60 test images, each accompanied by a Pascal VOC format XML annotation file containing one or more bounding boxes labelled apple, banana, or orange.

Two categories of source images were identified within the raw dataset: single-fruit images (filenames prefixed apple_, banana_, or orange_) and multi-fruit images (filenames prefixed mixed_), the latter containing two or more fruits — and therefore two or more annotated objects — in a single photograph, closely resembling a realistic “fruit basket” scene.

3.1.1 Annotation Format and Conversion

The raw annotations use the Pascal VOC XML format, specifying bounding boxes as absolute pixel coordinates (xmin, ymin, xmax, ymax). Because the YOLO-family detector used in this study (Section 3.3) requires normalised, centre-based coordinates, a conversion function was implemented to parse each XML file and transform every bounding box into the YOLO label format:

```
<class_id> <x_center> <y_center> <width> <height>   (all values normalised to [0, 1])
```

During this conversion, a data quality issue was identified and resolved: a subset of single-fruit XML annotation files (e.g. apple_44.xml) contained invalid, zero-valued image dimensions in their <size> tag (<width>0</width><height>0</height>). Trusting these values would have produced errors and corrupted normalised coordinates for the affected images. This was resolved by reading the true image dimensions directly from each image file using the Python Imaging Library (PIL.Image.open().size) rather than relying on the XML metadata. This step was essential for producing valid training labels and represents a deliberate data-cleaning decision taken after inspecting the raw annotation files.

3.1.2 Class Filtering and Basket Image Separation

Only annotations labelled apple or orange were retained, to remain consistent with the two fruit classes used in the classification task; banana annotations were excluded. Images containing no apple or orange instance at all (i.e. pure banana images) were discarded entirely. This filtering reduced the original 300 annotated images to 184 valid single-fruit images and 25 multi-fruit (“mixed”) images containing at least one apple or orange.

The 25 mixed-fruit images were deliberately withheld from the train/validation/test split and reserved exclusively as a qualitative “fruit basket” test set, since they represent realistic, cluttered, multi-object scenes that the quantitative train/test split does not capture. The remaining 184 single-fruit images were shuffled (seed=42) and split 70/15/15, producing the distribution shown in Table 3.1.

Split	Number of Images	Purpose
Train	128	Model training
Validation	27	Hyperparameter / epoch monitoring
Test	29	Final quantitative evaluation

Basket (held out)	25	Qualitative multi-fruit visualisation

Table 3.1: Object detection dataset distribution.

3.1.3 Directory Structure

```
detection_processed/
  images/
    train/   (128 images)
    val/     (27 images)
    test/    (29 images)
  labels/
    train/   (128 YOLO-format .txt files)
    val/     (27 .txt files)
    test/    (29 .txt files)
  basket_images/  (25 multi-fruit images + labels, qualitative test set)
  fruit_data.yaml (YOLO dataset configuration)
```

3.2 Object Detection Strategy

To satisfy the requirement of comparing detectors across architectural families, three detectors were selected, trained, and evaluated under identical conditions (same 128-image training set, 30 epochs, Tesla T4 GPU on Google Colab):

- YOLOv8n — single-stage, anchor-free detector (Ultralytics implementation)
- SSD-Lite (MobileNetV3-Large backbone) — single-stage, anchor-based detector (TorchVision implementation)
- Faster R-CNN (ResNet50-FPN backbone) — two-stage, region-proposal-based detector (TorchVision implementation)

This selection satisfies the project requirement to compare a Two-stage detector, a Single-stage detector, and a YOLO-family detector. All three were trained on identically formatted data (YOLO-normalised bounding boxes, converted to each library's required input format at load time) to ensure a fair, controlled comparison.

3.3 Results Visualisation

Model	Architecture Type	mAP@50	mAP@50-95	Recall	Inference Speed (FPS)	Training Time (30 epochs)	Parameters
YOLOv8n	Single-stage (anchor-free)	0.9841	0.7578	0.9520	24.8	~2 min	~3.0M
SSD-Lite (MobileNetV3)	Single-stage (anchor-based)	0.9251	0.5502	0.6307	149.0	~2-3 min	~3.4M
Faster R-CNN (ResNet50)	Two-stage	0.9737	0.7398	0.7740	5.9	~26 min	~41M

Table 3.2: Comparative test-set performance of the three object detection models.

3.4.1 YOLOv8n

YOLOv8n was trained for 30 epochs using the Ultralytics framework. On the held-out test set (29 images, 53 annotated instances), the model achieved an mAP@50 of 0.984 and an mAP@50-95 of 0.758, with precision and recall of 0.952 respectively. Training showed some instability mid-run where precision dropped sharply to 0.492 at epoch 11 before recovering. This is likely caused by the model's aggressive default data augmentation interacting with the very small 128-image training set. Inference speed averaged 40.3 ms per image (~24.8 FPS) including pre/post-processing overhead, comfortably within real-time deployment thresholds.

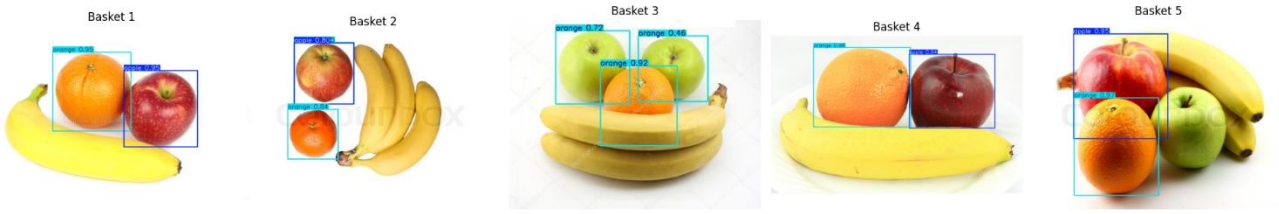

Figure 3.3: YOLOv8n detection results on the qualitative 'fruit basket' test images, with bounding boxes, class labels, and confidence scores.

Qualitative inspection of the basket results revealed a notable failure case: green apple instances were misclassified as “Orange” with moderate-to-high confidence (0.46–0.91) in several basket images. This is discussed further in Section 4.

3.4.2 SSD-Lite (MobileNetV3)

The SSD-Lite model, using a MobileNetV3-Large backbone pre-trained on ImageNet, was trained for 30 epochs using a custom PyTorch training loop. Training loss decreased smoothly and monotonically from 7.69 to 0.27 with no instability. On the test set, SSD achieved an mAP@50 of 0.925 and an mAP@50-95 of 0.550, with a mean recall of 0.631, noticeably lower than YOLOv8 on the stricter mAP@50-95 metric. Inference speed, however, was dramatically faster: 6.7 ms per image (~149 FPS), roughly six times faster than YOLOv8.

During evaluation, a UserWarning was raised indicating more than 100 detections were generated for a single image, requiring manual confidence thresholding (>0.3) to suppress excessive duplicate, overlapping bounding boxes. This indicates that SSD's built-in Non-Maximum Suppression is less aggressively tuned than YOLOv8's, and would require additional post-processing calibration for production/ daily application use.

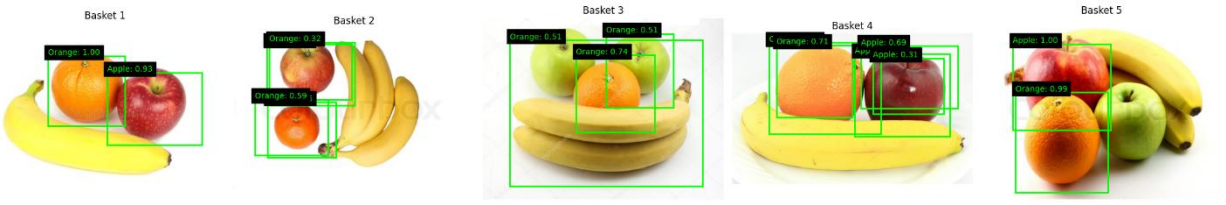

Figure 3.4: SSD-Lite detection results on the qualitative 'fruit basket' test images.

Qualitative inspection again revealed the same green-apple-to-orange misclassification pattern observed with YOLOv8 (Basket images 3 and 4), suggesting this is a dataset-level limitation rather than an architecture-specific one (see Section 4).

3.4.3 Faster R-CNN (ResNet50-FPN)

The Faster R-CNN model, using a ResNet50 backbone with a Feature Pyramid Network and pre-trained COCO weights (with the classification head replaced for 2 target classes plus background), was trained for 30 epochs. Training loss decreased smoothly from 0.52 to 0.033, the most stable convergence of the three detectors, though each epoch took approximately 51 seconds , roughly 13 times longer per epoch than YOLOv8 or SSD, reflecting the substantially larger ResNet50-FPN backbone (~41 million parameters versus ~3 million for the lightweight detectors).

On the test set, Faster R-CNN achieved an mAP@50 of 0.9737 and an mAP@50-95 of 0.739 , close to YOLOv8's mAP@50-95 score , with a mean recall of 0.774. Inference speed was the slowest of the three models at 195.91 ms per image (~5.9 FPS), unsuitable for real-time deployment.

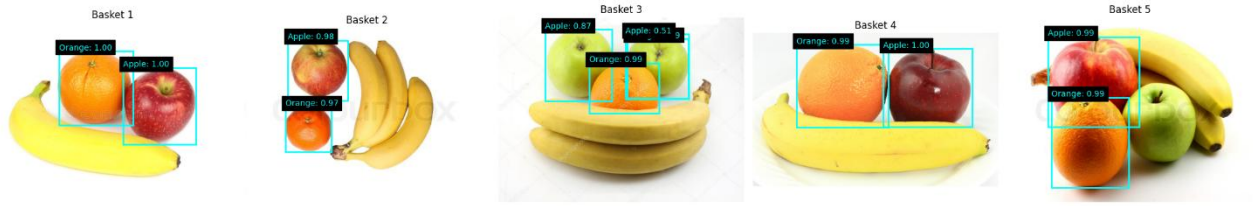

Figure 3.5: Faster R-CNN detection results on the qualitative 'fruit basket' test images.

Critically, Faster R-CNN was the only one of the three detectors to correctly classify the green apple instances in basket image 3, labelling them “Apple” rather than “Orange”. This qualitative difference, despite Faster R-CNN not having the highest quantitative mAP score, is a key finding discussed in Section 4.

4. Critical Comparative Analysis and Discussion

4.1 Accuracy vs. Speed Trade-offs

As summarised in Table 3.2 (reproduced for reference in Figure 4.1) and visualised in Figure 4.1, a clear accuracy-speed trade-off emerged across the three detectors.

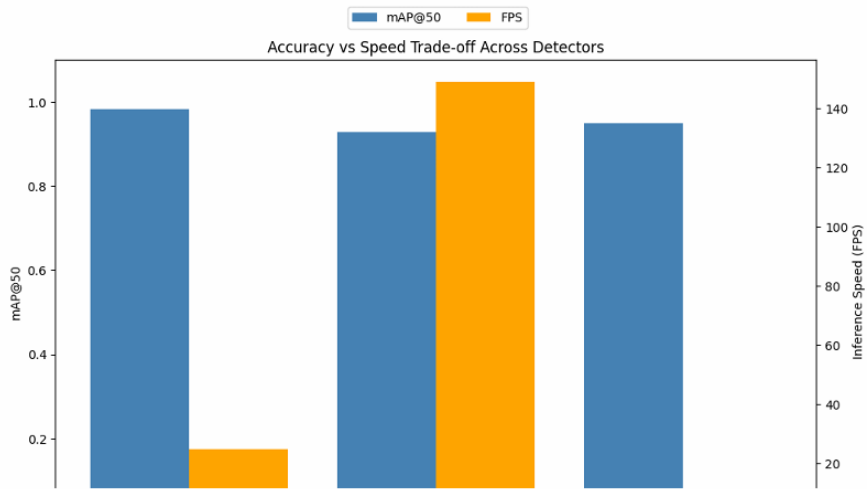

Figure 4.1: Accuracy (mAP@50) vs. Inference speed (FPS) trade-off across the three detectors.

YOLOv8n achieved the best overall balance, combining the highest mAP@50 (0.984) and recall (0.952) with a moderate inference speed (24.8 FPS), comfortably within real-time thresholds (typically above 15–20 FPS for video-rate applications). Its single-pass, anchor-free architecture allows it to localise and classify objects in one forward pass, avoiding the computational overhead of a separate region-proposal stage.

SSD-Lite was by far the fastest detector (149 FPS, approximately six times faster than YOLOv8), reflecting its lightweight MobileNetV3 backbone designed for mobile and edge deployment. However, this speed advantage came at a steep cost to robustness: its mAP@50-95 (0.55) and recall (0.63) were substantially lower than the other two models, and it required manual confidence thresholding to suppress excessive duplicate detections.

Faster R-CNN achieved the most competitive mAP@50-95 score (0.739, close to YOLOv8's 0.758) but was drastically slower (5.9 FPS, around twenty-five times slower than SSD), consistent with its two-stage, region-proposal-then-classify architecture.

The most important finding, however, came from qualitative rather than quantitative evaluation. Despite YOLOv8 achieving the highest mAP score, basket testing revealed that both YOLOv8 and SSD consistently misclassified green apple instances as oranges, while Faster R-CNN correctly classified them. This suggests that Faster R-CNN's two-stage region proposal mechanism extracts more robust shape- and texture-based features, whereas the single-stage models appear to rely more heavily on colour cues alone , a consequence of the training set being dominated by red apple instances, leaving the green apple variant underrepresented. This represents a genuine accuracy-robustness versus speed trade-off: the two-stage detector generalises better to underrepresented visual variants at the cost of being substantially slower at inference.

Implications for deployment: For real-time applications such as a conveyor-belt sorting system or a live camera feed, YOLOv8 is the most practical choice, meeting real-time speed requirements while retaining the highest accuracy. For edge or mobile deployment with strict latency constraints, SSD-Lite's speed advantage may justify. For offline batch for example, a quality-control auditing pipeline, Faster R-CNN's superior robustness to underrepresented visual variants makes it the safer choice despite its slow inference.

4.2 Model Complexity and Training Difficulty

Model	Parameters	Training Time (30 epochs)	Time / Epoch	Convergence Behaviour
YOLOv8n	~3.0M	~2 min	~4 sec	Instability mid-training (precision dropped to 0.49 at epoch 11) before recovering by epoch ~24
SSD-Lite	~3.4M	~2-3 min	~4-5 sec	Smooth, monotonic loss decrease (7.69 → 0.27), no instability
Faster R-CNN	~41M	~26 min	~51 sec	Smoothest convergence overall (0.52 → 0.033), but ~13x slower per epoch

Table 4.1: Model complexity and training behaviour across the three detectors.

Faster R-CNN's ResNet50-FPN backbone (approximately 41 million parameters) is roughly 12–14 times larger than YOLOv8n or SSD-Lite (approximately 3 million parameters each), directly explaining its longer training time per epoch despite all three models training on the identical 128-image dataset.

YOLOv8 showed the most volatile training curve of the three, with a sharp precision dip at epoch 11 before recovering by epoch 30 , likely due to its more aggressive default augmentation pipeline interacting with the very small dataset. SSD and Faster R-CNN, trained using simpler custom loops without built-in augmentation, showed smoother but comparatively slower convergence.

All three models trained successfully on a single Google Colab T4 GPU within a reasonable time for this small dataset, indicating none require specialised infrastructure at this scale. However, Faster R-CNN's substantially longer training time would become a more significant practical constraint at larger dataset scales or with frequent retraining cycles. The lightweight models (YOLOv8n, SSD-Lite) are more practical for rapid iterative experimentation, whereas Faster R-CNN's higher parameter count may offer better scalability to larger, more complex datasets where its additional representational capacity could be more fully exploited, the 128-image training set used in this study may be too small to fully leverage Faster R-CNN's capacity.

4.3 Final Recommendation

Based on the accuracy-speed trade-off analysis (Section 4.1) and the complexity/training difficulty analysis (Section 4.2), YOLOv8n is recommended as the most suitable model for a general-purpose, real-world fruit detection application, for the following evidence-based reasons:

- Best overall accuracy: highest mAP@50 (0.984) and recall (0.952) among all three models on the held-out test set.
- Real-time capable: at 24.8 FPS, it comfortably meets real-time deployment thresholds, unlike Faster R-CNN (5.9 FPS).
- Practical training cost: trains in approximately 2 minutes for 30 epochs, making it far more practical for iterative development and retraining as new fruit varieties are added, compared to Faster R-CNN's 26-minute training time.
- Fixable limitation: while YOLOv8 shares the green-apple misclassification issue with SSD, this is attributed to underrepresentation of green apples in the training data rather than a fundamental architectural flaw.

If the deployment context specifically prioritises maximum robustness over speed, for example, a quality-control checkpoint where misclassifying rare fruit variants is costly and inference can be batch-processed offline, then Faster R-CNN's superior handling of the green-apple edge case would make it the more defensible choice despite its computational cost. The final model choice should ultimately be guided by the specific deployment scenario's tolerance for speed versus robustness trade-offs.

5. Conclusion

This study implemented and compared three image classification models (SVM, Custom CNN, MobileNetV2) and three object detection models (YOLOv8n, SSD-Lite, Faster R-CNN) for an Apple vs. Orange computer vision task. All classification models achieved perfect accuracy, a result attributed to the controlled, low-noise conditions of the Fruits-360 dataset rather than evidence of true real-world robustness. The object detection comparison revealed a clear architectural trade-off between accuracy, speed, and robustness to underrepresented visual variants, with YOLOv8n offering the best overall balance for real-time deployment, while Faster R-CNN demonstrated superior robustness on a specific edge case (green apple misclassification) at a significant cost to inference speed.